parse_fastq.py 사용법 설명서

> lab11 · 생물정보학실험 대상 파일: `src/parse_fastq.py`

---

1. 프로그램의 목적

`parse_fastq.py`는 **FASTQ 파일을 읽어서 read(시퀀싱 조각)들을 하나씩 꺼내는 프로그램**입니다.

시퀀싱 기계는 DNA를 짧은 조각(read)으로 읽고, 그 결과를 FASTQ 파일로 저장합니다. FASTQ 파일은 수십만~수백만 줄이라 눈으로 보기 어렵기 때문에, 이 프로그램은 다음 네 가지를 간단히 알려줍니다.

- 파일 안에 read가 **몇 개** 있는지
- **첫 번째 read의 이름**
- **첫 번째 read의 염기서열**
- **첫 번째 read의 길이**

즉, "이 FASTQ 파일이 제대로 받아졌는지, 어떤 데이터인지"를 빠르게 확인하는 용도입니다.

---

2. 실행 명령어와 사용 방법

실행 형식

```
python3 src/parse_fastq.py 〈FASTQ 파일 경로〉
```

예시 (lab11 폴더에서 실행)

```
cd lab11
python3 src/parse_fastq.py data/SRR1553607_1.fastq
python3 src/parse_fastq.py data/SRR1553607_2.fastq
```

주의할 점

- 파일 경로를 **반드시 하나** 적어야 합니다. 빼먹으면 `sys.argv[1]`이 없어서 `IndexError`가 납니다.
- 압축된 파일(`.fastq.gz`)은 바로 읽을 수 없습니다. 먼저 압축을 풀어야 합니다.
- 프로그램은 파일이 **정확히 4줄씩** 되어 있다고 가정합니다. 파일이 중간에 잘려 있으면 에러가 날 수 있습니다.

---

3. 주요 코드의 역할

전체 코드는 다음과 같습니다.

```
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()

    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records

reads = read_fastq(sys.argv[1])
print("read 개수:", len(reads))
print("첫 read 이름:", reads[0][0])
print("첫 read 서열:", reads[0][1])
print("첫 read 길이:", len(reads[0][1]))
```

한 줄씩 역할을 정리하면 다음과 같습니다.

코드	역할
`import sys`	명령줄에서 입력한 파일 이름(`sys.argv`)을 쓰기 위해 불러옵니다.
`def read_fastq(filename):`	FASTQ 파일을 읽는 함수를 만듭니다.
`records = []`	꺼낸 read들을 담을 빈 리스트입니다.
`with open(filename) as f:`	파일을 엽니다. `with`를 쓰면 다 읽은 뒤 자동으로 닫힙니다.
`lines = f.readlines()`	파일의 **모든 줄**을 리스트로 읽어옵니다. `lines[0]`이 첫 줄, `lines[1]`이 둘째 줄…
`for i in range(0, len(lines), 4):`	4줄씩 건너뛰며 반복합니다. (4번 항목에서 자세히 설명)
`name = lines[i].rstrip()[1:]`	1번째 줄(이름 줄)에서 줄바꿈을 지우고(`rstrip()`), 맨 앞 `@`를 뺍니다(`[1:]`).
`seq = lines[i + 1].rstrip()`	2번째 줄 = 염기서열입니다.
`qual = lines[i + 3].rstrip()`	4번째 줄 = 품질 점수입니다. (3번째 줄 `+`는 쓸모가 없어서 건너뜁니다.)
`records.append((name, seq, qual))`	세 정보를 **튜플** 하나로 묶어 리스트에 추가합니다.
`return records`	모든 read가 담긴 리스트를 돌려줍니다.

코드	역할
`reads = read_fastq(sys.argv[1])`	명령줄에서 받은 파일 이름으로 함수를 실행합니다.
`len(reads)`	read의 개수입니다.
`reads[0][0]`	첫 번째 read(`[0]`)의 이름(`[0]`)
`reads[0][1]`	첫 번째 read(`[0]`)의 서열(`[1]`)
`len(reads[0][1])`	첫 번째 read 서열의 글자 수 = 길이

인덱스 정리 그림

```
reads = [
  ( 이름, 서열, 품질 ),   ← reads[0]  (첫 번째 read)
  ( 이름, 서열, 품질 ),   ← reads[1]
  ...
]
        [0]   [1]   [2]
```

---

4. `range(0, len(lines), 4)`의 의미

`range(시작, 끝, 간격)`은 **시작부터 끝 직전까지, 간격만큼 건너뛰며** 숫자를 만듭니다.

```
range(0, 12, 4)   # → 0, 4, 8
```

FASTQ 파일은 **read 하나가 정확히 4줄**입니다. 그래서 4씩 건너뛰면 `i`는 항상 **각 read의 첫 줄(@ 이름 줄)**을 가리킵니다.

i 값	lines[i]	lines[i+1]	lines[i+2]	lines[i+3]
0	read 1의 이름	read 1의 서열	`+`	read 1의 품질
4	read 2의 이름	read 2의 서열	`+`	read 2의 품질
8	read 3의 이름	read 3의 서열	`+`	read 3의 품질
…	…	…	…	…

- `len(lines)`는 전체 줄 수입니다. 이번 데이터는 **813,780줄**이므로 `i`는 0, 4, 8, …, 813,776까지 갑니다.
- 반복 횟수 = 813,780 ÷ 4 = **203,445번** = read 개수와 같습니다.

만약 간격을 1로 했다면 (`range(len(lines))`) 서열 줄이나 품질 줄을 이름으로 잘못 읽게 됩니다.

---

5. FASTQ 파일 구조와 read 추출 원리

FASTQ 한 레코드(= read 1개)는 4줄

실제 `data/SRR1553607_1.fastq`의 처음 4줄입니다.

```
@SRR1553607.1 1 length=101                          ← 1줄: @ + read 이름
GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAG...ACGACTCA   ← 2줄: 염기서열 (A, C, G, T, N)
+SRR1553607.1 1 length=101                          ← 3줄: + (구분선, 이름이 반복되기도 함)
BB@FFFFFHHHHHJJJJJJJJJJJJJJJJJJJJGHIJJ...DDDDDDDDD   ← 4줄: 품질 점수 (서열과 길이가 같음)
```

줄	시작 문자	내용
1	`@`	read의 이름(ID)과 설명
2	-	염기서열
3	`+`	구분선 (의미 없음)
4	-	품질 점수. 서열의 **글자 하나당 품질 문자 하나**

품질 점수 읽는 법 (참고)

품질 문자는 컴퓨터 문자 번호(ASCII)에서 33을 뺀 값이 품질 점수(Phred score)입니다.

문자	ASCII	품질 점수	의미
`#`	35	2	매우 나쁨
`5`	53	20	틀릴 확률 1/100
`?`	63	30	틀릴 확률 1/1,000
`J`	74	41	매우 좋음

```
ord("J") - 33   # → 41
```

FASTA와 무엇이 다른가?

	FASTA	FASTQ
이름 줄 시작	`>`	`@`
서열 줄 수	여러 줄 가능	보통 1줄
품질 정보	없음	있음
레코드 줄 수	정해져 있지 않음	**항상 4줄**

FASTQ는 레코드가 항상 4줄이라서, `>`를 찾아가며 읽는 FASTA와 달리 **줄 번호만으로** 이름·서열·품질을 꺼낼 수 있습니다. 이것이 이 프로그램의 핵심 원리입니다.

> ⁉왜 `@`로 시작하는 줄을 찾지 않고 4줄씩 자를까? 품질 줄도 `@` 문자로 시작할 수 있기 때문입니다 (실제로 `@`는 품질 31을 뜻함). 그래서 `@`로 찾으면 품질 줄을 이름으로 착각할 수 있어, 줄 위치로 자르는 방법이 더 안전합니다.

---

6. 실제 실행 결과와 해석

실행 1: `SRR1553607_1.fastq`

```
$ python3 src/parse_fastq.py data/SRR1553607_1.fastq
read 개수: 203445
첫 read 이름: SRR1553607.1 1 length=101
첫 read 서열:
GTTAGCGTTGTTGATCGCGACGCAACAACTGGTAAAGAATCTGGAAGAAGGATATCAGTTCAAACGCTCAAGCGAGATGATGGATATTTTTGAACGACTCA
첫 read 길이: 101
```

실행 2: `SRR1553607_2.fastq`

```
$ python3 src/parse_fastq.py data/SRR1553607_2.fastq
read 개수: 203445
첫 read 이름: SRR1553607.1 1 length=101
첫 read 서열:
TCATTCGTCACATCCCGTTCAACCTGATTGAGTCGTTCAAAAATATCCATCATCTCGCTTGAGCGTTTGAACTGATATCCTTCTTCCAGATTCTTTACCAG
첫 read 길이: 101
```

결과 해석

항목	값	의미
read 개수	203,445	파일 전체 줄 수 813,780 ÷ 4 = 203,445. 줄 수가 4의 배수이므로 **파일이 잘리지 않고 온전함**을 알 수 있습니다.
첫 read 이름	`SRR1553607.1 1 length=101`	`SRR1553607` = NCBI SRA 실행 번호, `.1` = 첫 번째 read, `length=101` = 길이 101 bp
첫 read 길이	101	이름에 적힌 `length=101`과 실제 서열 길이가 **일치**합니다. 이 시퀀싱은 101 bp짜리 read를 만든 실험입니다.

`_1`과 `_2` 파일은 왜 두 개일까? (paired-end)

- 두 파일은 read 개수(203,445)와 read 이름이 **똑같습니다**.
- 이것은 **paired-end 시퀀싱** 데이터이기 때문입니다. 같은 DNA 조각을 **양쪽 끝에서** 한 번씩 읽어서, 한쪽은 `_1`, 반대쪽은 `_2`에 저장합니다.
- 따라서 `_1`의 n번째 read와 `_2`의 n번째 read는 **짝(pair)** 입니다. 이름이 같은 이유도 그 때문입니다.
- 서열이 서로 다른 것은 같은 조각의 **서로 반대쪽 끝**을 읽었기 때문입니다.

처음 4줄의 품질 해석

첫 read의 품질 줄은 `BB@FFFFFHHHHHJJJJ...`로 시작합니다. `J`(41점), `H`(39점), `F`(37점) 같은 높은 점수가 많아 **품질이 좋은 read**라고 볼 수 있습니다. 뒤쪽으로 갈수록 `D`(35점) 정도로 조금 낮아지는데, 이는 시퀀싱 기계가 read 끝부분으로 갈수록 정확도가 떨어지는 일반적인 현상입니다.

부록: 이 코드의 한계 (더 알아보기)

한계	설명
메모리 사용	`readlines()`로 파일 전체를 한 번에 읽습니다. 이번 파일(약 56 MB)은 문제없지만, 수 GB짜리 파일은 메모리가 부족할 수 있습니다.
형식 검사 없음	첫 줄이 `@`로 시작하는지, 서열과 품질 길이가 같은지 확인하지 않습니다.
빈 파일	read가 0개면 `reads[0]`에서 `IndexError`가 납니다.
압축 파일	`.gz` 파일은 `gzip.open()`을 써야 읽을 수 있습니다.